



Stack



Data Structures and Algorithms with C++



Introduction to Stack

Currently we can store data using Array, LinkedList and Queue.

1. The array has the size constraint. The access of element is $a[i]$. In the array we can move in forward direction and opposite backward direction. In the array to insert at any position we need to shift. You can add/insert an element everywhere.
2. The Linked List does not have a size constraint. To access an element we need to iterate over the linked list. In a Singly linked list, we can only move only in forward direction. To move in both direction, you need to use the doubly linked list.

In the linked list there is no shift when we want to insert at a given position, there is no shifting, we need to change the links.

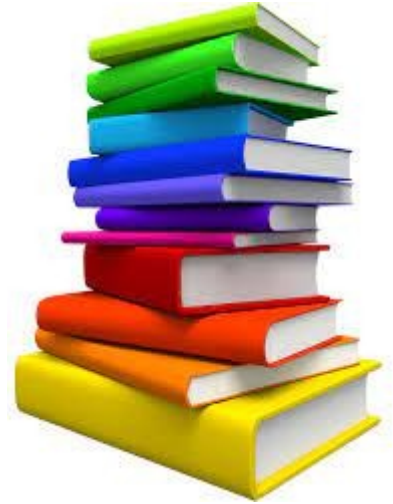
In the linked list, you can add or delete an element everywhere.

3. The Queue is an abstract data type which means there is a specific order in adding, deleting and accessing an element. The queue has not size constraint, but we need to add at the rear/tail and remove an element at the head or the front. The Queue can be implemented using Array or it can be implemented as a singly-linked list with a head and tail pointer.

Stack

In the Linked list or the array you can add or remove an element everywhere.

But consider the following scenario of books on the right. To remove a book, you need to start with the book on the top, then the second to most and so on. To add a book you need to add at the top.



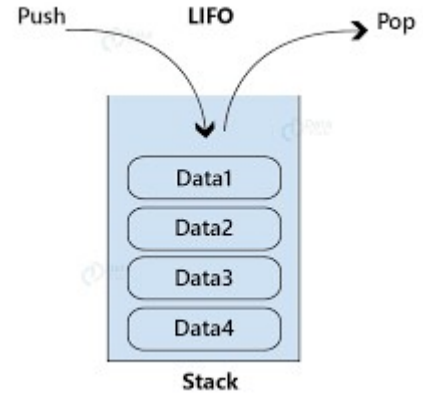
In this scenario of chairs, you can add a chair at the top, you can remove a chair at the top. You can not remove a chair in middle as well nor at the bottom. You can neither add a chair in the middle nor at the bottom.

Define a stack and supported Operations

A stack is a linear data structure that follows the principle of Last In First Out (LIFO).

This means the last element inserted inside the stack is removed first..

1. **Push:** Add an element to the top of a stack
2. **Pop:** Remove an element from the top of a stack
3. **IsEmpty:** Check if the stack is empty
4. **IsFull:** Check if the stack is full
5. **Top/Peak:** Get the value of the top element without removing it.
6. **Size:** Get the number of elements in the stack



Recursion uses stack

```
fact(0)
fact(1)
fact(2)
fact(3)
fact(4)
fact(5)
```

Consider the recursive factorial function of a number 5 in memory. fact(5) will call fact(4) then call fact(3) then call fact(2) then call fact(1) then call fact 0.

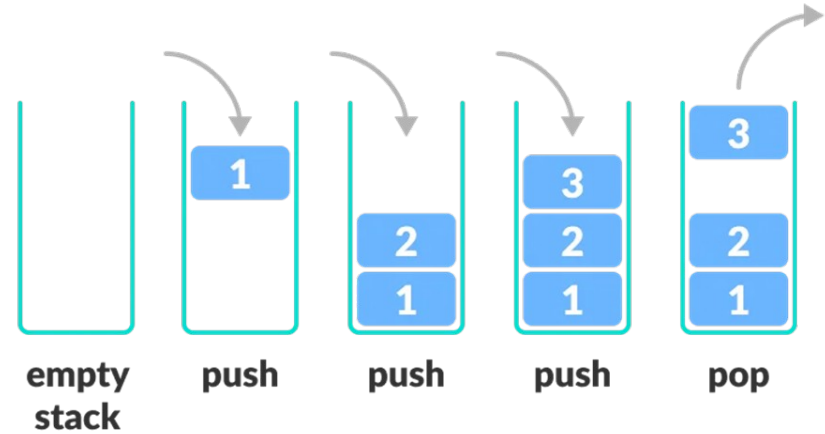
In memory, this will be a stack represented as per left illustration.

In the execution the factorial of 5 will be started by the factorial of zero, then factorial of 1, the factorial of 2.

LIFO principle of a stack

In programming terms, putting an item on top of the stack is called push and removing an item is called pop/peek.

In the following , although item 3 was kept last, it was removed first. This is exactly how the **LIFO (Last In First Out)** Principle works.



Stack implementation using Array

We can implement a stack using Array or linked list. Here we are going to start with the use of Arrays.

Consider the following array `arr[5]` of 5 elements. To insert an element we need to push the element at the **nextIndex** starting from **zero** and keep the **nextIndex** as 1.

Push(10), nextIndex=1
Push(20), nextIndex=2
Push(30), nextIndex=3
Push(40), nextIndex=4

0	1	2	3	4	5
10	20	30	40		

40
30
20
10

To get the element at the top is to use `top()`, means accessing `arr[nextIndex-1]`
In our case, it is `arr[4-1]` which means `arr[3]`
To remove an element `pop(40)` means `nextIndex--` which means next `push(35)` will override the value 40.
The `size()` will return `nextIndex`. `isEmpty` will return true if the `nextIndex` is zero.

Implementation using array: Working of a stack data structure

Using arrays, the operations work as follows:

1. Using array we need to keep track of the **nextIndex** to track the element in the stack.
2. When initializing the stack, we set **nextIndex** value to 0 so that we can check if the stack is empty by comparing **nextIndex == 0**.
3. On pushing an element, we add an element at the **nextIndex**, then we increase/change the value of **nextIndex**.
4. On popping an element, we return the element available at **arr[nextIndex -1]** and reduce its value.

Algorithm for PUSH operation

5. Check if the stack is full or not.
6. If the stack is full, then print an error of overflow and exit the program.
7. If the stack is not full, add the element and increment the top

Algorithm for POP operation

8. Check if the stack is empty or not.
9. If the stack is empty, then print an error of underflow and exit the program.
10. If the stack is not empty, then print the element at the top and decrement the top.

Implementation using Array

```
#include<iostream>
using namespace std;
class Stack{
    int *arr;
    int nextIndex;
    int capacity;
public:
    Stack(){
        capacity = 4;
        arr = new int[capacity];
        nextIndex = 0;
    }
    Stack(int cap){
        capacity = cap;
        arr = new int[capacity];
        nextIndex = 0;
    }
    /// return no of elements present in stack
    int size(){ return nextIndex;}
    bool isEmpty(){
        /*if(nextIndex==0){
            return true;
        }else{
            return false;
        }*/
        return nextIndex==0;
    }
}
```

```
void push(int ele){
    if(nextIndex==capacity){
        cout<<"Stack full"<<endl;
        return;
    }
    arr[nextIndex] = ele;
    nextIndex++;
}
void pop(){
    if(isEmpty()){
        cout<<"Stack empty"<<endl;
        return;
    }
    nextIndex--;
}
int top(){
    if(isEmpty()){
        cout<<"Stack empty"<<endl;
        return 0;
    }
    return arr[nextIndex-1];
}
};
```

```
int main(){
    Stack s(4);
    s.push(10);
    s.push(20);
    s.push(30);
    s.push(40);
    s.push(50);

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.size()<<endl;
    cout<<s.isEmpty()<<endl;

    Stack s2(10);
    for(int i=1;i<=10;i++){
        s2.push(i);
    }
    /// print content of stack
    while(!s2.isEmpty()){
        cout<<s2.top()<<endl;
        s2.pop();
    }
    cout<<s2.size()<<endl;
    return 0;}
```


Dynamic Array: Implementation of stack

We can remove the constraint of the array by doubling the capacity of the array whenever the stack is full and copy all data into new Array and then delete the old array. We change parameterised constructor and the push function.

```
#include<iostream>
using namespace std;

class Stack{
    int *arr;
    int nextIndex;
    int capacity;

public:
    Stack(){
        capacity = 4;
        arr = new int[capacity];
        nextIndex = 0;
    }
```

```
void push(int ele){
    if(nextIndex==capacity){
        int *newArr = new int[2*capacity];
        for(int i=0;i<capacity;i++){
            newArr[i] = arr[i];
        }
        delete []arr;
        arr = newArr;
        capacity = 2*capacity;
    }
    arr[nextIndex] = ele;
    nextIndex++;
}
```

```
int main(){
    Stack s;
    s.push(10);
    s.push(20);
    s.push(30);
    s.push(40);
    s.push(50);

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.size()<<endl;
    cout<<s.isEmpty()<<endl;
    return 0;
}
```

Review Questions

1. A stack is
 - a. FIFO(First In First Out) list
 - b. LIFO(Last In First Out) list
 - c. Ordered Array
 - d. Linear Tree
2. Which data structure is used for implementing recursion?
 - a. Queue
 - b. Stack
 - c. Linked list
 - d. Array
3. Which of the following statement(s) about stack data structure is NOT correct?
 - a. Stack Data Structure can be implemented using a linked list
 - b. New Node can be only added at the top
 - c. Stack is the FIFO data structure
4. If the elements "p", "q", "r" and "s" are placed in a stack and are deleted one at a time, in what order will they be removed?
 - a. srqp
 - b. pqrs
 - c. Srpq
 - d. qrsp
6. 'When Array implementation of Stack is not dynamic', which of the following statements supports this argument?
 - a. Space allocation for the array is fixed and can not be changed during run-time.
 - b. User unable to give the input for stack operations
 - c. A run time execution error
 - d. All the mentioned
7. In order to keep track of current topmost element of the stack(using array implementation) we need to maintain one variable.
 - A. True
 - B. False
8. What is the time complexity of pop() operation when the stack is implemented using an array?
 - C. $O(1)$
 - D. $O(n)$
 - E. $O(\log n)$
 - F. $O(n \log n)$
9. Reverse a String Using a Stack.
10. Check for Balanced Parentheses.

Generics in C++

Generics is the idea to allow type (Integer, String, ... etc and user-defined types) to be a parameter to methods, classes and interfaces. For example, classes like an array, map, etc, which can be used using generics very efficiently. We can use them for any type.

The method of Generic Programming is implemented to increase the efficiency of the code. Generic Programming enables the programmer to write a general algorithm which will work with all data types. It eliminates the need to create different algorithms if the data type is an integer, string or a character.

The advantages of Generic Programming are

1. Code Reusability
2. Avoid Function Overloading
3. Once written it can be used for multiple times and cases.

Generics can be implemented in C++ using template. Template is a simple and yet very powerful tool in C++. The simple idea is to pass data type as a parameter so that we don't need to write the same code for different data types. For example, a software company may need sort() for different data types. Rather than writing and maintaining the multiple codes for integer, we can write one sort() and pass data type as a parameter.

Example2: Generic Function using Template

Generic Functions using Template:

We write a generic function that can be used for different data types. Examples of function templates are `sort()`, `max()`, `min()`, `printArray()`

```
#include <iostream>
using namespace std;

// One function works for all data types.
// This would work even for user defined types
// if operator '>' is overloaded
template <typename T>
T myMax(T x, T y)
{
    return (x > y) ? x : y;
}

int main()
{
    // Call myMax for int
    cout << myMax<int>(3, 7) << endl;
    // call myMax for double
    cout << myMax<double>(3.0, 7.0) << endl;
    // call myMax for char
    cout << myMax<char>('g', 'e') << endl;
    return 0;
}
```

Example3: Generic class using Template

Generic Class using Template:

Like function templates, class templates are useful when a class defines something that is independent of data type. Can be useful for classes like LinkedList, binary tree, Stack, Queue, Array, etc.

```
#include<iostream>
using namespace std;

template<typename T>
class Pair{
    T x;
    T y;

public:
    void setX(T x){
        this->x = x;
    }
    T getX(){
        return x;
    }
    void setY(T y){
        this->y = y;
    }
    T getY(){
        return y;
    }
};
```

```
int main(){

    Pair<int> p1;
    p1.setX(1);
    p1.setY(2);
    cout<<p1.getX()<<" "<<p1.getY()<<endl;
    Pair<double> p2;
    p2.setX(1.73);
    p2.setY(2.21);
    cout<<p2.getX()<<" "<<p2.getY()<<endl;

    Pair<char> p3;
    Pair<int,double> p1;
    p1.setX(1.73);
    p1.setY(1.73);
    cout<<p1.getX()<<" "<<p1.getY()<<endl;

    return 0;
}
```

Example4: Multiple Generics

Multiple generics

A class can define multiple
Generics in C++

```
#include<iostream>
using namespace std;

template<typename T,
typename V>
class Pair{
    T x;
    V y;

    public:
        void setX(T x){
            this->x = x;
        }
        T getX(){
            return x;
        }
        void setY(V y){
            this->y = y;
        }
        V getY(){
            return y;
        }
};
```

```
int main(){

    Pair<int,double> p1;
    p1.setX(1.73);
    p1.setY(1.73);

    cout<<p1.getX()<<"
"<<p1.getY()<<endl;

    return 0;
}
```

Example 5: Template Usage with Nesting

```
#include<iostream>
using namespace std;
template<typename T, typename V>
class Pair{
    T x;
    V y;
public:
    void setX(T x){
        this->x = x;
    }
    T getX(){
        return x;
    }
    void setY(V y){
        this->y = y;
    }
    V getY(){
        return y;
    }
};
```

In the main function below,
we pass T as also a Pair
taking T, V datatype and V
as a third datatype.

```
int main(){
    Pair<Pair<int,int> , int> p;
    p.setY(30);
    Pair<int,int> temp;
    temp.setX(10);
    temp.setY(20);
    p.setX(temp);
    cout<<p.getX().getX()<<" "<<p.getX().getY()<<" "<<p.getY()<<endl;
    return 0;
}
```

Stack implementation using Template

In previous slides we reviewed the use of Templates. Additionally we created a Stack of Integers. In the next section we are going to enable our Stack to store different data types like integers, double, characters

Stack with Generic Template

```
#include<iostream>
using namespace std;

template<typename T>
class Stack{
    T *arr;
    int nextIndex;
    int capacity;

public:
    Stack(){
        capacity = 4;
        arr = new T[capacity];
        nextIndex = 0;
    }
    /// return no of elements present in
    stack
    int size(){
        return nextIndex;
    }
    bool isEmpty(){

        return nextIndex==0;
    }
}
```

```
void push(T ele){
    if(nextIndex==capacity){
        T *newArr = new T[2*capacity];
        for(int i=0;i<capacity;i++){
            newArr[i] = arr[i];
        }
        delete []arr;
        arr = newArr;
        capacity = 2*capacity;
    }
    arr[nextIndex] = ele;
    nextIndex++;
}

void pop(){
    if(isEmpty()){
        cout<<"Stack empty"<<endl;
        return;
    }
    nextIndex--;
}

T top(){
    if(isEmpty()){
        cout<<"Stack empty"<<endl;
        return 0;
    }
    return arr[nextIndex-1];
}

};
```

```
int main(){
    Stack<char> s;
    s.push(100);
    s.push(101);
    s.push(102);
    s.push(103);
    s.push(104);

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.size()<<endl;
    cout<<s.isEmpty()<<endl;
    return 0;
}
```

Stack Implementation using Linked List

Introduction

To implement a_stack using the singly linked list concept, all the singly_linked list operations should be performed based on Stack operations LIFO(last in first out) and with the help of that knowledge, we are going to implement a stack using a singly linked list.

So we need to follow a simple rule in the implementation of a stack which is **last in first out** and all the operations can be performed with the help of a top variable.

Insert at tail or Maintaining a head and tail pointer

Insert at tail:

10,20,30,40

10->20->30->40->NULL

You need to change the head with head=head->next until the last node is found.

To insert you need to iterate until you reach the tail pointer which makes the time complexity $O(n)$.

Maintaining head and tail pointer

```
Class stack{  
Node *head;  
Node *tail;  
}
```

With push function we can make push constant

If we have a linked list: 10->20->30->40->50->NULL

To add a new Node with 60 we need to create the new Node and connect with the tail which is constant $O(1)$.

If 10->20->30->40->50->60->NULL

For pop(), the last node with data **60** should be removed and the new **tail** will be a node with **50**. Using singly linked list, we will delete the tail pointer but we face the problem of **updating the tail pointer** since we can not move in backward direction.

top() can be implemented using tail->data

isEmpty() can be implemented checking if head =NULL

size() by maintaining the variable size and increment as we push and decrement when pop is called.

Using the head

Suppose we have elements 10,20,30,40,50 and we want to insert at head:

1. **Push()**

- Initially the head is NULL and first data is 10.
- 10->NULL and is the new head node
- Adding new node with 20 and we need to point to the head node and change the head to the new node.
- 20->10->NULL. Now 20 is in the head node.
- To add a new node with 30, point to 20 and change the head to 30
- 30->20->10->NULL
- Now push will be constant $O(1)$.

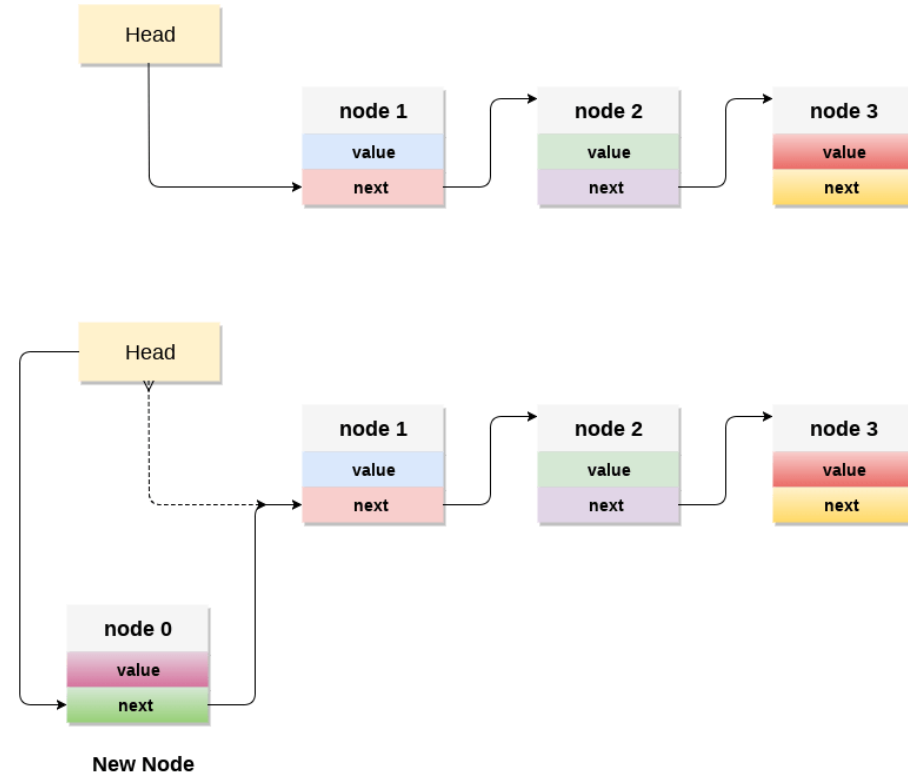
1. **Pop()** we will keep the head in temp, update the **head** pointer to the next of head, isolate temp by changing its next to NULL and delete temp
2. **top()** should return head->data
3. **size()** should return the value of size variable.
4. **isEmpty()** should return head==NULL

Adding a node to the stack (Push operation) with Visual representation

In order to push an element onto the stack using linked list, the following steps are involved.

If the list is empty then the item is to be pushed as the start node of the list. This includes assigning value to the data part of the node and assign null to the address part of the node.

If there are some nodes in the list already, then we have to add the new element in the beginning of the list. For this purpose, assign the address of the starting element to the address field of the new node and make the new node, the starting node of the list.



Pop()-Deleting the Node

Deleting a node from the top of stack is referred to as **pop** operation. Deleting a node from the linked list implementation of stack is different from that in the array implementation. In order to pop an element from the stack, we need to follow the following steps :

1. **Check for the underflow condition:** The underflow condition occurs when we try to pop from an already empty stack. The stack will be empty if the head pointer of the list points to null.
2. **Adjust the head pointer accordingly:** In stack, the elements are popped only from one end, therefore, the value stored in the head pointer must be deleted and the node must be freed. The next node of the head node now becomes the head node.

Summary of Stack Operations

Push Operation:

- Initialise a node
- Update the value of that node by data i.e. **node->data = data**
- Now link this node to the top/head of the linked list
- And update top/head pointer to the current node
- Update by incrementing the size variable,

Pop Operation:

- First Check whether there is any node present in the linked list or not, if not then return
- Otherwise make pointer let say **temp** to the top/head node and move forward the top/head node by 1 step
- Simultaneously print the value of the temp node
- Now free this temp node
- Update the size variable by decrementing it

Peek Operation:

- Check if there is any node present or not, if not then return.
- Otherwise return the value of top node of the linked list

Sample program

```
#include<iostream>
using namespace std;

template <typename T>
class Node{
    T data;
    Node<T> *next;
public:
    Node(T data){
        this->data = data;
        next = NULL;
    }
};
```

```
template<typename T>
class Stack{
    Node<T> *head;
    int size;
public:
    Stack(){
        head = NULL;
        size = 0;
    }
};
```

```
bool isEmpty(){

}

void push(T ele){

}

void pop(){

}

T top(){

}

};

int main(){
    Stack<int> s;
    return 0;
}
```

Complete Program

```
#include<iostream>
using namespace std;
```

```
template <typename T>
class Node{
public:
    T data;
    Node<T> *next;
    Node(T data){
        this->data = data;
        next = NULL;
    }
};
```

```
template<typename T>
class Stack{
    Node<T> *head;
    int size;
public:
    Stack(){
        head = NULL;
        size = 0;
    }
    int getSize(){
        return size;
    }
    bool isEmpty(){
        return size==0;
    }
    void push(T ele){
        Node<T> *n = new Node<T>(ele);
        n->next = head;
        head = n;
        size++;
    }
```

```
void pop(){
    if(isEmpty()){
        cout<<"Stack empty"<<endl;
        return;
    }
    Node<T> *temp = head;
    head = head->next;
    temp->next = NULL;
    delete temp;
    size--;
}

T top(){
    if(isEmpty()){
        cout<<"Stack empty"<<endl;
        return 0;
    }
    return head->data;
};
```

```
int main(){
    Stack<int> s;
    s.push(100);
    s.push(101);
    s.push(102);
    s.push(103);
    s.push(104);

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.getSize()<<endl;
    cout<<s.isEmpty()<<endl;
    return 0;
}
```

Complete Program without Generic

```
#include<iostream>
using namespace std;
```

```
class Node{
public:
    int data;
    Node *next;
    Node(int data){
        this->data = data;
        next = NULL;
    }
};
```

```
class Stack{
    Node *head;
    int size;
public:
    Stack(){
        head = NULL;
        size = 0;
    }
    int getSize(){
        return size;
    }
    bool isEmpty(){
        return size==0;
    }
    void push(int ele){
        Node *n = new Node(ele);
        n->next = head;
        head = n;
        size++;
    }
};
```

```
void pop(){
    if(isEmpty()){
        cout<<"Stack empty"<<endl;
        return;
    }
    Node *temp = head;
    head = head->next;
    temp->next = NULL;
    delete temp;
    size--;
}

int top(){
    if(isEmpty()){
        cout<<"Stack empty"<<endl;
        return 0;
    }
    return head->data;
};
```

```
int main(){
    Stack s;
    s.push(100);
    s.push(101);
    s.push(102);
    s.push(103);
    s.push(104);

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.top()<<endl;
    s.pop();

    cout<<s.getSize()<<endl;
    cout<<s.isEmpty()<<endl;
    return 0;
}
```

C++ Provide in built in stack

Functions of implemented stack	Inbuilt in stack
Void push()	Void push()
Void pop()	Void pop()
T top()	T top()
Int getSize()	Int size()
Bool isEmpty	Bool empty()

Inbuilt in stack use

```
#include<iostream>
#include<stack>
using namespace std;
```

```
int main(){
    stack<int> s;
    s.push(1);
    s.push(2);
    s.push(3);
    s.push(4);
    s.push(5);
    s.push(6);
```

— . — . — ➔

```
    cout<<s.top()<<endl;
    s.pop();
    cout<<s.top()<<endl;
    s.pop();

    cout<<s.size()<<endl;
    cout<<s.empty()<<endl;
    while(is.empty()){
        cout<<s.top()<<endl;
        s.pop();
    }
    cout<<s.size()<<endl;
    return 0;
}
```

Exercises

1. Consider the linked list implementation of a stack. Which of the following node is considered as Top of the stack?
 - a) First Node
 - b) Last Node
 - c) Any Node
 - d) Middle Node

2. In linked representation of stack, last node of linked list represents
 - e) Beginning of the stack
 - f) Bottom of the stack
 - g) Middle of the stack
 - h) In between some values

Exercises(next)

3. Balanced Parenthesis

Given a string `s` containing just the characters '(', ')', '{', '}', '[' and ']', write a function using stack to determine if the input string is valid. An input string is valid if:

- Open brackets must be closed by the same type of brackets.
- Open brackets must be closed in the correct order.
- Every close bracket has a corresponding open bracket of the same type.

`() [] {}` return true

`()` should return true

`[]` should return false

`([])` should return false

`{ [] }` should return false

`([ ] { } ( ) )` should return true

Rules:

1. If it is opening bracket, then push
2. Closing bracket, `]` the top bracket of the stack should be `[`, if the closing bracket is `}`, the top bracket of the stack should be `{`.

If the closing is not corresponding with the opening then return false.

3. If the closing bracket is corresponding with the opening top bracket then do `pop()`.
4. After traversing the string and the stack is empty then return true.

Sample Solution: Balanced Parentheses

```
#include<iostream>
#include<string>
#include<stack>
using namespace std;
class Parentheses {
public:
    bool isValid(string A) {
        stack<char> s;
        for(int i=0;i<A.size();i++){

            if(A[i]=='(' || A[i]=='{' || A[i]=='['){
                s.push(A[i]);
            } else{

                if(s.empty()) return false;
                else if(A[i]==')'){
                    if(s.top() == '(') s.pop();
                    else return false;
                }else if(A[i]=='}'){
                    if(s.top() == '{') s.pop();
                    else return false;
                }else if(A[i]==']'){
                    if(!s.empty() && s.top() == '[') s.pop();
                    else return false;
                }
            }
        }
        if(s.empty()) return true;
        return false;
    }
};
```

```
int main(){
    string sample="{}{}";
    Parentheses pr;
    cout<<pr.isValid(sample);
    return 0;

}
```


Exercise 4. Reverse Polish Notation (next)

Reverse Polish Notation (RPN) was devised as a method of simplifying mathematical expressions.

RPN predates modern computers. It became useful with the advent of electronic calculators that were not great at handling complex expressions that needed brackets. As such, there is no need for brackets in RPN, because the calculators allow expressions to be entered in unambiguous postfix form.

You are given a string that represents an arithmetic expression in a Reverse Polish Notation

Write a program with a function using stack to evaluate the expression. The function should return *an integer that represents the value of the expression*.

Note that:

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always **truncates toward zero**.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.

More info on RPN:

https://isaacomputerscience.org/concepts/dsa_toc_rpn?examBoard=all&stage=all

Example 1:

Input: tokens = {"2", "1", "+", "3", "*"}

Output: 9

Explanation: $((2 + 1) * 3) = 9$

Example 2:

Input: tokens = {"4", "13", "5", "/", "+"}

Output: 6

Explanation: $(4 + (13 / 5)) = 6$

RPN:Sample Solution

```
#include<iostream>
#include<vector>
#include<stack>
using namespace std;
class Rpn {
public:
    int evalRPN(vector<string>& tokens) {
        stack<int> st;

        for(int i=0;i<tokens.size();i++){

            if(tokens[i]=="+" || tokens[i]=="-" || tokens[i]=="*" || tokens[i]=="/"){
                // 2 elements
                int v1 = st.top();
                st.pop();
                int v2 = st.top();
                st.pop();

                if(tokens[i]==""){
                    st.push(v2+v1); // push result inside stack
                }
                else if(tokens[i]=="-"){
                    st.push(v2-v1); // push result inside stack
                }
                else if(tokens[i]=="*"){
                    st.push(v2*v1); // push result inside stack
                }
                else if(tokens[i]=="/"){
                    st.push(v2/v1); // push result inside stack
                }
            }

            else {
                // operands
                st.push(stoi(tokens[i]));
            }
        }

        // one element inside the stack
        return st.top();
    }
};
```

```
int main(){
    Rpn rpn;
    vector<string> vect={"4","13","5","/","+"}; //(4 + (13 / 5)) = 6
    cout<<rpn.evalRPN(vect);
    return 0;
}
```

Analysis of Stack Operations

Below mentioned are the time complexities for various operations that can be performed on the Stack data structure.

- **Push Operation : $O(1)$**
- **Pop Operation : $O(1)$**
- **Top Operation : $O(1)$**
- **Search Operation : $O(n)$**

The time complexities for push() and pop() functions are $O(1)$ because we always have to insert or remove the data from the top of the stack, which is a one step process.

Applications of Stack Data Structures

Although stack is a simple data structure to implement, it is very powerful. The most common uses of a stack are:

- **To reverse a word** - Put all the letters in a stack and peek them out. Because of the LIFO order of stack, you will get the letters in reverse order.
- **In compilers** - Compilers use the stack to calculate the value of expressions like $2 + 4 / 5 * (7 - 9)$ by converting the expression to prefix or postfix form.
- **In browsers** - The back button in a browser saves all the URLs you have visited previously in a stack. Each time you visit a new page, it is added on top of the stack. When you press the back button, the current URL is removed from the stack, and the previous URL is accessed.